Biomedical Image Classification — Technical Walkthrough

Data → preprocessing → method → code → measurement → refinement, with the reasoning behind each decision.

DermaMNIST · ResNet-50 · RTX 4050 6 GB · every figure from a recorded run in `results/`

---

1. The data: what it actually is

1.1 Provenance chain

```
HAM10000 (Tschandl et al. 2018)
  10,015 dermatoscopic images, Vienna (Austria) + Queensland (Australia)
  ground truth: histopathology (~50%), follow-up, expert consensus, confocal microscopy
        |
        v  downsample to 28x28, fixed 7:1:2 split
DermaMNIST  (MedMNIST v2, Yang et al. 2023)
        |
        +-- default:      28x28    18.8 MB
        +-- MedMNIST+:    224x224   1041 MB   <- what this project switched to
```

1.2 What one item literally is

```
ds = DermaMNIST(split="train", root="./data")
img, lab = ds[0]

img : PIL.Image, mode="RGB", size=(28, 28)     # already 3-channel
lab : numpy.ndarray, shape=(1,), value=[0]     # NOT a scalar
```

Two details that cause real bugs:

- **Labels arrive shaped `(1,)`, not as scalars.** A DataLoader batch is therefore `(B, 1)`, and passing that straight into `cross_entropy` either errors or silently broadcasts. Every label is flattened with `labels.view(-1).long()`.
- **DermaMNIST is already RGB** (`n_channels: 3`). The `to_rgb` conversion in the transform is a no-op here, but it is required for the greyscale MedMNIST datasets (PneumoniaMNIST, BreastMNIST) that share this pipeline, since ImageNet backbones demand 3 channels.

1.3 Splits and class distribution

Class	Meaning	train	val	test	% of test
nv	melanocytic nevi (benign mole)	4,693	—	1,341	66.9%
mel	**melanoma (malignant)**	779	—	223	11.1%
bkl	benign keratosis-like	769	—	220	11.0%
bcc	basal cell carcinoma	359	—	103	5.1%
akiec	actinic keratoses / intraepithelial ca.	228	—	66	3.3%
vasc	vascular lesions	99	—	29	1.4%
df	dermatofibroma	80	—	23	1.1%
total		**7,007**	**1,003**	**2,005**	

Imbalance ratio 4693 : 80 ≈ 58 : 1. This single number drives the loss design, the metric choice, the noise floor, and most of the failure modes below.

> **Why the splits are never re-shuffled.** MedMNIST ships fixed official train/val/test partitions and this project uses them unchanged. Two reasons: results stay comparable with every published MedMNIST benchmark, and re-splitting risks leaking test images into training. `load_split()` is the only entry point and takes the split name as an argument, so there is no code path that could resplit by accident.

2. Data problems you must know about

2.1 Resolution — the dominant one

28×28 = **784 pixels**. The network consumes 224×224 = **50,176 pixels**. The default pipeline bridges that with `Resize((224,224))` — an 8× linear, 64× areal upsample that manufactures no information. Dermoscopic diagnosis depends on pigment networks, dots and globules, streaks and blue-white veil; none of that structure exists at 28×28. Section 5 measures what it costs: **0.21 macro-F1**.

2.2 Population skew

HAM10000 was collected in Austria and Queensland — both predominantly light-skinned populations. Fitzpatrick V–VI skin is severely underrepresented. A dermatology model trained on this should be assumed *not* to generalise across skin tones, and MedMNIST carries no demographic metadata, so the bias is **unmeasured, not absent**.

2.3 Possible lesion-level leakage UNVERIFIED

HAM10000 contains multiple photographs of the *same physical lesion* — roughly 10,015 images across ~7,500 distinct lesions. If MedMNIST partitioned by *image* rather than by *lesion*, near-duplicate views of one lesion can appear in both train and test, inflating every score. This has not been verified here and should be, before quoting any figure as generalisation performance.

2.4 Tiny classes make metrics unstable

`df` has 23 test images. One image changing outcome moves its recall by 4.3 points, and because macro-F1 weights all classes equally, that alone moves macro-F1 by ~0.6 points. This is measured directly in section 13.

3. Preprocessing, step by step

Defined once in `evaluate.py` and imported by `train.py` and `app.py`. Duplication here is how training and evaluation silently diverge.

```
transforms.Compose([
    transforms.Resize((224, 224)),               # 1
    transforms.Lambda(to_rgb),                   # 2
    transforms.ToTensor(),                       # 3
    transforms.Normalize(mean=[0.485,0.456,0.406],
                         std =[0.229,0.224,0.225]) # 4
])
```

#	Step	What it does	Why
1	`Resize`	28×28 → 224×224 (bilinear)	ResNet-50's ImageNet pretraining assumes ~224px inputs. At 224px native source this is a no-op.
2	`to_rgb`	ensures 3 channels	No-op for DermaMNIST; required for greyscale datasets sharing this pipeline.
3	`ToTensor`	HWC uint8 [0,255] → CHW float [0,1]	PyTorch convolutions expect channel-first float tensors.
4	`Normalize`	(x − mean) / std, per channel	Matches the input statistics ResNet-50 was pretrained on. Skip it and the pretrained filters see a distribution they were never trained for.

Measured output: `torch.float32`, shape `(3, 224, 224)`, value range `[-0.390, 1.752]` — not [0,1], because normalisation recentres around zero. Seeing negative pixel values here is correct.

> **Why `to_rgb` is a named module-level function rather than a lambda.** Windows uses the *spawn* start method for DataLoader workers, which pickles the transform to send it to child processes. Lambdas are not picklable. A lambda here works fine at `num_workers=0` and crashes the moment anyone sets `num_workers=4` — a bug that only appears under a performance flag.

4. Augmentation

Applied to the **training split only**, and in this recipe only for DermaMNIST.

```
RandomHorizontalFlip()
RandomVerticalFlip()
RandomRotation(30)
ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)
```

Why these specific transforms are valid here

A transform is only legitimate if it **preserves the label**. That is a domain question, not a generic one:

- **Flips and rotations are safe for dermoscopy** because a skin lesion has no canonical orientation. The dermatoscope can be held any way round; a lesion rotated 180° is the same lesion.
- **They would not be safe for chest X-rays.** A horizontal flip puts the heart on the right — dextrocardia, a real medical condition. The same augmentation that regularises one modality fabricates pathology in another.

> **Colour jitter is the risky one, and it is applied here.** Melanoma diagnosis depends substantially on *colour variegation* — multiple distinct pigment shades within one lesion is a malignancy criterion. Jittering brightness, contrast and saturation attacks exactly the signal being classified. At 0.2 it is mild; pushing it to 0.4 may cost more than it buys. Testing colour and geometric augmentation separately remains an unrun backlog item.

> **The rule that matters:** augmentation on train, never on val or test. With `augment=False` the training transform is byte-identical to the evaluation transform. If augmentation leaked into evaluation, every metric would be measuring a randomly perturbed image and would not be reproducible.

5. The resolution decision

The single highest-impact change in the project, and it is a *data* decision, not a modelling one.

	28px (default)	224px (MedMNIST+)
file size	18.8 MB	1,041 MB
source pixels	784	50,176
tensor into network	3×224×224	3×224×224
GPU compute	identical	identical
VRAM	identical	identical
sec/epoch (mean, 3 seeds)	67.8	75.7

Timing is noisy: 224px runs ranged 56.8–113.1 s/epoch across seeds, one of them *faster* than 28px. That spread reflects machine contention, not workload. Treat per-epoch timing as approximate; total wall-clock rose ~24% mainly because 224px runs trained for more epochs before early stopping.

> **The key insight to be able to state:** the network received 224×224 tensors in *both* conditions. Identical FLOPs, identical memory. The 28px pipeline was performing an expensive upsample to manufacture pixels that carried no information. Switching to native resolution was not "spending more compute for accuracy" — it was *stopping the discarding of data that was already available*.

6. Architecture

Model	Parameters	Role
ResNet-50	23,522,375	teacher / main model
DenseNet-121	6,961,031	teacher
EfficientNet-B0	4,016,515	student

6.1 The head swap

```
model = resnet50(weights=IMAGENET1K_V1)   # 1000-class ImageNet head
model.fc = nn.Linear(2048, 7)             # replaced: 14,343 params
```

The final layer maps a 2048-dimensional feature vector to 7 class logits. Everything before it is reused. Note the proportion: **14,343 new parameters against 23.5 million inherited** — 0.06%. Essentially the entire network is pretrained knowledge.

6.2 All layers are trainable

This recipe fine-tunes every parameter from step one at a uniform learning rate. That is a real choice with a real trade-off: maximum adaptation to the new domain, but 23.5M parameters being updated by 7,007 images, and pretrained early-layer structure being overwritten by gradients from a small dataset. Discriminative learning rates and progressive unfreezing are the standard alternatives and remain untested here.

7. The loss function, built up in layers

The recipe's loss is four ideas stacked. Understanding it means separating them.

Layer 1 — Cross-entropy

```
CE = -log( p[correct class] )
```

The baseline. Problem: with 4,693 `nv` training images against 80 `df`, the gradient is overwhelmingly `nv`. The model minimises total loss efficiently by getting `nv` right and ignoring rare classes.

Layer 2 — Class weights

Multiply each sample's loss by a per-class weight so rare classes contribute more. The recipe computes sklearn `balanced` weights then damps them three ways:

```
w = balanced_weights        # spread 58 : 1
w = w ** 0.5                # spread sqrt(58) = 7.6 : 1
w = w / w.mean()            # pure rescale, spread unchanged
w = clamp(w, 0.1, 2.0)      # <-- never binds
```

Class	n (train)	final weight
df	80	1.873
vasc	99	1.684
akiec	228	1.110
bcc	359	0.884
bkl	769	0.604
mel	779	**0.600**
nv	4,693	0.245

> **Two findings worth stating in an interview.**
>
> **(a) The clamp is dead code.** After the square root and mean-normalisation the maximum weight is 1.873 — no class ever reaches the 2.0 ceiling. The clamp looks like a safety mechanism and does nothing. The square root alone does all the damping.
>
> **(b) Melanoma is *under*-weighted.** At `0.600`, the clinically critical class receives below-average weight, because with 779 images it is not rare enough to trigger the correction. The scheme optimises for *rarity*, not for *clinical cost* — and those are not the same objective.

Layer 3 — Focal loss

```
FL = (1 − p_t)^gamma · CE          gamma = 1.5
```

When the model is already confident and correct (`p_t → 1`), the factor `(1 − p_t)^1.5 → 0` and that example nearly stops contributing gradient. Training concentrates on hard examples. With thousands of easy `nv` images, this is directly targeting the imbalance from a second angle.

Layer 4 — Label smoothing

```
target = 0.9 for true class, 0.1/(K−1) spread over the rest
```

Discourages overconfidence and slightly improves calibration.

> **The design flaw:** the original implementation hardcodes `label_smoothing=0.1` *inside* the `FocalLoss` class. Three distinct techniques — weighting, focal scaling, smoothing — are welded into one object and cannot be varied independently. No one can say which is helping, which is inert, and which is hurting.
>
> The rewritten `train.py` exposes `--focal-gamma`, `--label-smoothing` and `--loss {auto,focal,ce}` as separate flags, defaults unchanged. The ablation is now *possible*; it has not yet been run.

8. Optimiser, AMP, and the training loop

Setting	Value	Reasoning
optimiser	Adam	Robust default; adapts per-parameter step sizes, tolerant of poor LR choice.
learning rate	1e-4, constant	Low, because fine-tuning pretrained weights. A high LR destroys inherited features. No schedule — cosine annealing is an untested backlog item.
batch size	64	110 steps/epoch. Fits 6 GB with AMP; at ~5–5.5 GB it is near the limit.
precision	AMP (fp16 autocast)	Roughly halves activation memory and speeds up matmuls on Ada tensor cores. Without it, batch 64 at 224px would not fit.
max epochs	50	Ceiling only; early stopping always triggered first (11–20 epochs observed).

8.1 Why `GradScaler` is required with AMP

fp16 has a much narrower dynamic range than fp32. Small gradients underflow silently to zero, and the affected weights simply stop learning with no error. `GradScaler` multiplies the loss by a large factor before `backward()`, pushing gradients into fp16's representable range, then divides them out before the optimiser step — and skips any step where it detects inf/NaN.

> **A bug found in the original:** one global `GradScaler` was shared across all nine training runs, carrying its adapted loss-scale state between unrelated models. Each run now constructs its own.

8.2 One training step

```
optimizer.zero_grad(set_to_none=True)          # set_to_none frees memory vs zeroing
with autocast("cuda"):                         # forward in fp16
    loss = criterion(model(images), labels)
scaler.scale(loss).backward()                  # scale up, then backprop
scaler.step(optimizer)                         # unscale, skip if inf/NaN
scaler.update()                                # adapt scale factor
```

9. Early stopping

Track validation loss each epoch. If it fails to improve for `patience=5` consecutive epochs, stop and **restore the weights from the best epoch**.

WHY The model overfits well before epoch 50. Measured on seed 42 at 28px: training loss fell 0.395 → 0.222 while validation loss bottomed at 0.300 and then rose — a final train/val gap of +0.118. Without early stopping you would ship the epoch-50 weights, which are strictly worse.

> **But it is optimising the wrong quantity.** Validation loss on DermaMNIST is dominated by `nv` at 67% of the split. Observed on every seed examined, the epoch with the best validation *macro-F1* was rejected in favour of the epoch with the best validation *loss*:
>
> | Seed | epoch kept (best loss) | epoch discarded (best macro-F1) | macro-F1 given up |
> |---|---|---|---|
> | 42 | 6 → 0.5569 | 7 → 0.5767 | −0.0198 |
> | 1337 | 13 → 0.5829 | 9 → 0.6021 | −0.0192 |
>
> `--early-stop-metric val_macro_f1` is implemented; the experiment has not been run, so no gain is claimed.

9.1 Why best weights are snapshotted to CPU

The snapshot is `{k: v.detach().cpu().clone()}`. Keeping it on the GPU would hold a second full ResNet-50 in VRAM alongside the training graph, optimiser state and activations. On a 6 GB card already running at ~5–5.5 GB, that is an out-of-memory error — and it would appear at the *first improving epoch*, typically epoch 1, not at the end.

10. Knowledge distillation

```
Loss = alpha · CE(student, true labels)
     + (1 − alpha) · T² · KL( softmax(student/T) || softmax(teacher/T) )

alpha = 0.7,  T = 3.0
```

WHY A one-hot label says "this is melanoma" and nothing else. A teacher's full distribution — "80% melanoma, 15% nevus, 5% keratosis" — additionally says *melanoma and nevus look similar here*. That relational information is called **dark knowledge**, and hard labels destroy it.

Temperature `T` divides logits before softmax, flattening the distribution so those small probabilities become visible in the gradient. The `T²` factor compensates for the fact that softening shrinks gradient magnitudes by roughly `1/T²`, keeping the two loss terms on a comparable scale as `T` changes.

11. Code structure

```
evaluate.py   <-- owns the shared definitions
  IMAGENET_MEAN/STD, build_eval_transform(), build_model(), load_split(), to_rgb()
  predict(model, loader, device, tta)      metrics, confusion matrix, AUC, metrics.json
        ^                    ^
        | imports            | imports
train.py                  app.py
  FocalLoss                 Flask: /health /classes /predict /explain
  EarlyStopping             GradCAM (context-managed hooks)
  compute_class_weights()
  training loop -> checkpoint + history.json

tune_thresholds.py  <-- reads saved .npz probabilities, fits decision rules on val
```

11.1 Why preprocessing and model construction live in one place

If `train.py` normalised with `[0.5,0.5,0.5]` while `evaluate.py` used the ImageNet constants, nothing would crash. There would be no error message. Evaluation would simply feed the network a distribution it was never trained on, and every reported metric would be quietly wrong — a degraded but plausible-looking number. This is the classic **train/serve skew** bug, and a single shared definition is the only reliable defence.

11.2 The checkpoint carries its own identity

```
{ state_dict, model_name, dataset_flag, num_classes, label_names,
  img_size, medmnist_size, seed, run_name, epochs_trained, best_epoch,
  train_seconds, hyperparams{...} }
```

A bare `state_dict` does not say which architecture or dataset produced it. With a dozen Phase-2 checkpoints, mislabelling one silently corrupts the experiment log. `evaluate.py` reads this metadata rather than trusting CLI flags, and hard-errors if the checkpoint's class count disagrees with the dataset's.

11.3 Three engineering rules earned from bugs

- **Never let a presentation step destroy a computation step.** An early version wrote `metrics.json` *after* rendering the confusion matrix. When Windows Application Control blocked matplotlib's `_backend_agg` DLL, the process died and a completed evaluation was lost. Rendering is now wrapped; failures become warnings.
- **Omit what you cannot compute; never substitute a placeholder.** scikit-learn returns `NaN` (with a *warning*, not an exception) for macro AUC when a class has no positives. Such metrics are omitted and noted in a `warnings` array, and `json.dumps(allow_nan=False)` makes any non-finite value a hard error.
- **Scope resources to their use.** The original Grad-CAM registered hooks on every call and never removed them, so they accumulated on the model. Now context-managed, with a regression test asserting the hook count is unchanged after repeated calls.

12. Metrics: definitions and traps

```
precision = TP / (TP + FP)      "when it says X, how often is it right"
recall    = TP / (TP + FN)      "of all real X, how many did it find"
F1        = 2PR / (P + R)       harmonic mean

macro-F1    = mean of per-class F1, every class equal
weighted-F1 = mean of per-class F1, weighted by support
balanced acc = mean of per-class recall
```

The calculation that explains the whole project. Replace the model with `return nv` — predict benign mole for every image:

```
accuracy = 1341/2005 = 0.6688

nv: precision 0.6688, recall 1.0000, F1 = 2(0.6688)(1)/(1.6688) = 0.8016
all six other classes: F1 = 0

macro-F1 = 0.8016 / 7 = 0.1145
```

67% accuracy, 0.11 macro-F1. A model that detects zero melanomas scores two-thirds accuracy. That is why this project reports macro-F1.

Why AUC can look excellent while F1 is poor

Baseline at 28px: macro AUC **0.9334**, macro-F1 **0.5587**. AUC is threshold-free — it measures whether the model *ranks* positives above negatives. F1 measures the *decisions* produced by `argmax`. A model can rank almost perfectly and still decide badly, because under a 67% majority prior `argmax` demands overwhelming evidence before predicting a rare class. That gap looked like free headroom, and section 14 tests it — twice, with negative results both times.

13. Measurement discipline

13.1 The noise floor (E0)

Three runs, identical configuration, seed the only difference.

Seed	macro-F1	weighted-F1	accuracy	epochs
42	0.5224	0.7450	0.7332	11
1337	0.5944	0.7588	0.7451	18
2024	0.5594	0.7561	0.7392	13
mean ± sd	**0.5587 ± 0.0360**	0.7533 ± 0.0073	0.7392 ± 0.0060	

Macro-F1 is 5–6× noisier than the metrics originally reported.

- The original single-seed baseline (0.5224) was the *worst* of the three.
- An unpaired single-run comparison needs **> ~0.07 macro-F1** to mean anything. Most published techniques are worth 0.01–0.03 — individually undetectable this way.
- The prior claim that a distilled student beat its teacher by **0.65 accuracy points** sits inside a seed-only range of **1.20 points**: roughly half the noise of its own measurement.

Per-class variance shows where the noise comes from:

Class	n	s42	s1337	s2024	range
vasc	29	0.4602	0.8070	0.6067	**0.347**
df	23	0.3684	0.5333	0.3614	**0.172**
bkl	220	0.4239	0.4916	0.5280	0.104
nv	1341	0.8736	0.8830	0.8726	0.010

13.2 Paired vs unpaired — the technique that rescues this

A **paired** experiment reuses the *same trained checkpoints* and changes only inference or the decision rule. Seed variance affects both sides identically and cancels, so effects an order of magnitude below the 0.036 unpaired floor become visible. TTA (+0.017) is only measurable because it is paired. Resolution is necessarily unpaired, but at +0.21 the distinction does not matter.

14. The experiments

EXP-4 — Native 224px data ADOPTED

Metric	28px	224px	Δ
macro-F1	0.5587 ± 0.0360	**0.7733 ± 0.0248**	**+0.2145**
weighted-F1	0.7533	0.8622	+0.1089
accuracy	0.7392	0.8589	+0.1197
macro AUC	0.9334	0.9771	+0.0437

Roughly six times the noise floor. Per-class gains follow the mechanism precisely — the classes needing fine texture gain most, the easy majority class gains least:

Class	n	28px	224px	Δ
bkl	220	0.4812	0.7572	+0.2760
akiec	66	0.4338	0.7082	+0.2744
df	23	0.4211	0.6904	+0.2693
vasc	29	0.6246	0.8813	+0.2567
bcc	103	0.5550	0.7838	+0.2288
mel	223	0.5191	0.6632	+0.1441
nv	1341	0.8764	0.9286	+0.0523

That differential is the evidence. If extra pixels were simply adding generic capacity, all classes would rise together. They do not.

EXP-1 / EXP-5 — Test-time augmentation KEPT

Average softmax over four views: original, h-flip, v-flip, both.

	28px	224px
Δ macro-F1	+0.0171	+0.0174
paired p (n=3)	0.119	0.081
positive seeds	3/3	3/3

Nearly identical at both resolutions, so the gains **stack**: TTA reduces decision variance, resolution adds information — different mechanisms. Cost: 4× inference (~9s → ~37s), no retraining, no extra VRAM.

Honest caveat: at n=3 the paired t-test does not reach conventional significance and the 95% CI (−0.011 to +0.045) includes zero. Consistent direction and sound mechanism, but promising rather than proven.

EXP-2 / EXP-6 — Logit adjustment REJECTED

Subtract `τ·log(train prior)` from each log-probability, `τ` fitted on validation. Expected to work because macro AUC 0.93 vs macro-F1 0.52 implied the decision rule was the weak link.

Result: validation selected `τ ≈ 0` on nearly every seed — the unadjusted rule was already optimal. Mean test Δ: −0.0028 (28px), +0.0012 (224px). No effect.

Explanation: the class-weighted focal loss already applies a prior correction *during training*. Applying it again at decision time double-counts.

EXP-3 — Per-class decision weights REJECTED

Resolution	validation Δ	test Δ
28px	+0.0566	+0.0033
224px	+0.0451	**−0.0105**

> **The clearest overfitting demonstration in the project.** Seven free parameters fitted by coordinate ascent against 1,003 validation images, four classes of which have <60 examples. It gains ~+0.05 macro-F1 on validation *every seed, at both resolutions*, and delivers nothing at 28px and active harm at 224px.
>
> Contrast with EXP-2's *single* parameter, which gained nothing anywhere. Together they isolate the cause: the 7-parameter rule did not fail because threshold tuning is wrong in principle — it failed because it had enough capacity to memorise validation noise.

Where the total gain came from

Configuration	macro-F1 (3 seeds)
28px, argmax — original recipe	0.5587 ± 0.0360
224px, argmax	0.7733 ± 0.0248
224px + TTA	**0.7906 ± 0.0314**

+0.2319 total: 92% input resolution, 8% test-time augmentation. Neither touched the model, the loss, the optimiser or the schedule. Both *rejected* experiments were decision-rule changes — the same conclusion from the other direction.

15. Error analysis — what the model actually confuses

Seed 42, 28px, the largest off-diagonal confusion cells:

True → Predicted	images	% of true class	reading
nv → mel	114	8.5%	benign called malignant — costly but safe direction
bkl → mel	45	20.5%	benign called malignant
nv → bkl	45	3.4%	benign ↔ benign
nv → vasc	44	3.3%	benign ↔ benign
bkl → nv	41	18.6%	benign ↔ benign
bkl → akiec	33	15.0%	benign called pre-cancerous
mel → nv	32	14.3%	**malignant called benign — the dangerous direction**

Moving to 224px collapses almost every confusion — **except the one that matters most**:

True → Predicted	28px	224px	change
nv → mel	114	31	−83
nv → vasc	44	2	−42
bkl → mel	45	6	−39
bkl → akiec	33	10	−23
mel → nv	32	53	**+21**

The most important negative finding in the project. On seed 42, higher resolution made the *one clinically dangerous error more common*. Melanomas classified as benign lesions (`nv` or `bkl`):

Seed	28px	224px
42	50/223 (22.4%)	**74/223 (33.2%)**
1337	64/223 (28.7%)	33/223 (14.8%)
2024	55/223 (24.7%)	43/223 (19.3%)

The mean improves (25.3% → 22.4%) but the variance is enormous and one seed got much worse. **Macro-F1 rose 0.21 while the most safety-critical error rate did not reliably improve at all.** An aggregate metric can rise while the thing you actually care about does not. This is precisely why per-class error analysis belongs alongside any headline number.

16. Why this method — the chain

1. **7,007 images is too few to train 23.5M parameters from scratch** → transfer learning from ImageNet.
2. **ImageNet backbones expect ~224px, 3-channel, ImageNet-normalised input** → that exact transform chain, defined once and shared.
3. **58:1 imbalance means plain cross-entropy ignores rare classes** → class weights, then focal loss attacking the same problem from a second angle.
4. **Imbalance also means accuracy and weighted-F1 are misleading** → macro-F1 as the target metric.
5. **Small data plus 23.5M parameters means fast overfitting** → early stopping with best-weight restoration, plus augmentation chosen for label-preservation in this modality.
6. **6 GB VRAM at batch 64, 224px** → AMP with GradScaler; CPU-side weight snapshots.
7. **Any claimed improvement must be distinguishable from luck** → seeding, then measuring the noise floor before running a single experiment.
8. **The noise floor (0.036) exceeds most techniques' effect sizes** → paired experimental designs wherever possible.
9. **Measured result: the data, not the method, was the binding constraint** → native 224px adopted; the loss-engineering backlog deprioritised accordingly.

17. Limitations and open questions

- **Melanoma remains unsolved.** Best recall ~0.72, unstable across seeds, and the malignant→benign error rate did not reliably improve with resolution.
- **Loss-component ablation never run.** Which of weighting / focal / smoothing helps is still unknown; the code now permits the test.
- **Six backlog experiments unrun** — cosine LR, discriminative LR with progressive unfreezing, class-weight power sweep, SWA, mixup, cutmix. Each needs ≥3 seeds to clear the noise floor.
- **Early-stop-on-macro-F1** implemented, untested.
- **Lesion-level leakage unverified** — if MedMNIST split by image rather than lesion, every number here is optimistic.
- **No calibration, no demographic evaluation.** Softmax outputs are not probabilities; bias across skin tones is unmeasured, not absent.
- **The Docker image has never been built** — written and reasoned through, but unverified.

> **One-sentence summary.** The original work tuned loss functions inside an artificial 28×28 constraint that cost 0.21 macro-F1 — more than any of those techniques could recover — and the way that was established was by measuring the noise floor first, so real effects could be told apart from luck.

All figures produced by `train.py` / `evaluate.py` / `tune_thresholds.py`, recorded in `results/`. Hardware: RTX 4050 Laptop 6 GB, Python 3.14, torch 2.13.0+cu126. Harness, experiments and analysis by Nitin Joshi.